Network Distributed Computing
Assignment 1
Kevin Ceausu – B00128171

1. How to run the project:

For running the project there are essentially 2 options, first you can open the project on Visual Studio Code and open a terminal where you launch the first server with the command **py s.py 1.** You can launch as many servers as you like by opening multiple terminals, for example in next terminal you can write **py s.py 2** in order to start the second server. Now you can open a new terminal and write py c.py in order to start the client. From there a menu list will open and you be will be asked to input the number of the function that you would like to use.

For the second option to run the project, I have a script .bat file which you can launch by clicking it in the assignment's directory. From there you will be asked how many servers you would like to open, enter your input and it will launch as many servers you entered along with the client.

I have also created a python unit test file which you can simply launch in the terminal with the command **py test_server.py**.
In that file there are unit tests for all the functions implemented in the project's code. **Very important, in order to successfully run the file and pass all the unit tests, it's crucial that you first launch server 1, server 2 and server 3 in 3 different terminals, only after that you can write py test_server.py in a new terminal and all unit tests can run properly.**

2. Client Python code (c.py):

```
from jsonrpcclient import request, parse, Ok
import requests

def call_ping():
  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("ping")
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")

def call_whoareyou():
  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("whoareyou")
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")

def call_get_version():
  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("get_version")
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")
```

```
def call_make_folder():
  print("what folder name?")
  foldername = input()

  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("make_folder", params={"foldername": foldername})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")

def call_delete_folder():
  print("what folder name?")
  foldername = input()

  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("delete_folder", params={"foldername": foldername})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")

def call_search():
  print("what filename to search?")
  filename = input()

  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("search", params={"filename": filename})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")
```

```
def call_list_friends():
  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("list_friends")
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")


def call_online():
  print("which server port is coming online?")
  onlinePort = int(input())

  print("send this command to which server?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("online", params={"port": onlinePort})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")


def call_offline():
  print("which server port is going offline?")
  offlinePort = int(input())

  print("send this command to which server?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("offline", params={"port": offlinePort})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")
```

```
def call_heartbeat():
  print("what is the servers port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("heartbeat")
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")


def call_startup():
  print("which server number should be started?")
  x = input()

  print("send this command to which server?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("startup", params={"x": x})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")


def call_shutdown():
  print("which server number should be shut down?")
  x = input()

  print("send this command to which server port?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("shutdown", params={"x": x})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
```

```
    print("Are you sure that port and function exist?")



def call_pass_message():
  print("what message do you want to send?")
  msg = input()

  print("which server number is the target?")
  x = input()

  print("send this command to which server port first?")
  portToCall = int(input())

  try:
    response = requests.post(
      f"http://localhost:{portToCall}/",
      json=request("pass_message", params={"msg": msg, "x": x})
    )
    parsed = parse(response.json())
    print(parsed.result)
  except:
    print("Are you sure that port and function exist?")



print("Welcome!")

while True:
  print("\nplease type a menu option")
  print("1. ping")
  print("2. whoareyou")
  print("3. get_version")
  print("4. make_folder")
  print("5. delete_folder")
  print("6. search")
  print("7. list_friends")
  print("8. online")
  print("9. offline")
  print("10. heartbeat")
  print("11. startup")
  print("12. shutdown")
  print("13. pass message")
  print("14. quit")

  option = int(input())

  if option == 1:
    call_ping()
  elif option == 2:
    call_whoareyou()
```

```
  elif option == 3:
    call_get_version()
  elif option == 4:
    call_make_folder()

  elif option == 5:
    call_delete_folder()

  elif option == 6:
    call_search()

  elif option == 7:
    call_list_friends()
  elif option == 8:
    call_online()
  elif option == 9:
    call_offline()
  elif option == 10:
    call_heartbeat()
  elif option == 11:
    call_startup()
  elif option == 12:
    call_shutdown()
  elif option == 13:
    call_pass_message()
  elif option == 14:
    break
```

3. Server python code (s.py):

```
from jsonrpcserver import Success, method, serve
import sys
import os
import requests
from jsonrpcclient import request, parse
import subprocess
import threading

# Get server number from command line
if len(sys.argv) > 1:
  serverNumber = sys.argv[1]
else:
  print("sorry you forgot to add a server number... shutting down")
  quit()

# List of known friend servers
myFriends = []

@method
def ping():
  return Success("pong")

@method
def whoareyou():
  portNumber = "500" + serverNumber
  return Success(f"I am server {serverNumber} running on port {portNumber}")

@method
def get_version():
  return Success(sys.version)

@method
def make_folder(foldername):
  try:
    os.mkdir(foldername)
    return Success(f"Folder '{foldername}' created")
  except FileExistsError:
    return Success("Folder already exists")


@method
def delete_folder(foldername):
  try:
    os.rmdir(foldername)
```

```
    return Success(f"Folder '{foldername}' deleted")
  except FileNotFoundError:
    return Success("Folder not found")
  except OSError:
    return Success("Folder not empty")


@method
def search(filename):
  files = os.listdir(".")
  if filename in files:
    return Success(f"{filename} found")
  else:
    return Success(f"{filename} not found")

@method
def list_friends():
  if len(myFriends) == 0:
    return Success("No friends known")
  return Success(", ".join(str(friend) for friend in myFriends))


@method
def online(port):
  port = int(port)

  if port not in myFriends:
    myFriends.append(port)

  # tell other friends this server is online too
  for friend in myFriends:
    if friend != port:
      try:
        requests.post(
          f"http://localhost:{friend}/",
          json=request("add_friend", params={"port": port})
        )
      except:
        pass

  return Success(f"Server on port {port} is now online")


@method
def add_friend(port):
  port = int(port)
  if port not in myFriends:
    myFriends.append(port)
  return Success("friend added")

@method
```

```
def offline(port):
  port = int(port)

  if port in myFriends:
    myFriends.remove(port)

  for friend in myFriends:
    try:
      requests.post(
        f"http://localhost:{friend}/",
        json=request("remove_friend", params={"port": port})
      )
    except:
      pass

  return Success(f"Server on port {port} is now offline")

@method
def remove_friend(port):
  port = int(port)
  if port in myFriends:
    myFriends.remove(port)
  return Success("friend removed")

@method
def heartbeat():
  if len(myFriends) == 0:
    return Success("No friends to ping")

  results = []

  for friend in myFriends:
    try:
      response = requests.post(
        f"http://localhost:{friend}/",
        json=request("ping")
      )
      parsed = parse(response.json())
      results.append(f"{friend}: {parsed.result}")
    except:
      results.append(f"{friend}: no response")

  return Success(" | ".join(results))


@method
def startup(x):
  x = str(x)
  newPort = int("500" + x)

  try:
```

```
    #start another server process
    subprocess.Popen([sys.executable, os.path.abspath(__file__), x])

    # tell this server / network that the new server is online
    if newPort not in myFriends:
      myFriends.append(newPort)

    for friend in myFriends:
      if friend != newPort:
        try:
          requests.post(
            f"http://localhost:{friend}/",
            json=request("add_friend", params={"port": newPort})
          )
        except:
          pass

    return Success(f"Started server {x} on port {newPort}")
  except Exception as e:
    return Success(f"Could not start server: {str(e)}")


@method
def shutdown(x):
  x = str(x)
  targetPort = int("500" + x)

  # tell all known friends that this target is offline
  if targetPort in myFriends:
    myFriends.remove(targetPort)

  for friend in myFriends:
    try:
      requests.post(
        f"http://localhost:{friend}/",
        json=request("remove_friend", params={"port": targetPort})
      )
    except:
      pass

#tell the target server to shut itself down
  try:
    requests.post(
      f"http://localhost:{targetPort}/",
      json=request("shutdown_self")
    )
    return Success(f"Shutdown signal sent to server {x}")
  except:
    return Success(f"Could not contact server {x} on port {targetPort}")
```

```
@method
def pass_message(msg, x, visited=None):
  if visited is None:
    visited = []

  targetServer = str(x)
  myPort = int("500" + serverNumber)

  if myPort in visited:
    return Success("Already visited this server")

  visited.append(myPort)

# if this server is the target
  if serverNumber == targetServer:
    return Success(f"Message received by server {serverNumber}: {msg}")

  #otherwise pass to friends
  for friend in myFriends:
    if friend not in visited:
      try:
        response = requests.post(
          f"http://localhost:{friend}/",
          json=request(
            "pass_message",
            params={"msg": msg, "x": x, "visited": visited}
          )
        )
        parsed = parse(response.json())
        return Success(parsed.result)
      except:
        pass

  return Success("Target server not found")


@method
def shutdown_self():
  def stop_server():
    os._exit(0)

  threading.Timer(0.5, stop_server).start()
  return Success("Server shutting down")


if __name__ == "__main__":
  print(f"server number {serverNumber} running.....")
  portNumber = "500" + serverNumber
  serve(port=int(portNumber))
```

4. Script used to run the python code (run_server.bat):

```
@echo off
set /p count=How many servers do you want to start? 

set i=1

:loop
if %i% GTR %count% goto done

start "Server%i%" cmd /k py s.py %i%

set /a i+=1
goto loop

:done
py c.py
pause
```

5. Python unit tests (test_server.py):

```
import unittest
import requests
import os
from jsonrpcclient import request, parse

SERVER1 = "http://localhost:5001/"
SERVER2 = "http://localhost:5002/"
SERVER3 = "http://localhost:5003/"


class TestServer(unittest.TestCase):

  def rpc_call(self, server_url, method_name, params=None):
    if params is None:
      response = requests.post(server_url, json=request(method_name))
    else:
      response = requests.post(server_url, json=request(method_name, 
params=params))
    parsed = parse(response.json())
    return parsed.result

  #BASIC COMMANDS

  def test_ping(self):
    result = self.rpc_call(SERVER1, "ping")
    self.assertEqual(result, "pong")

  def test_whoareyou(self):
    result = self.rpc_call(SERVER1, "whoareyou")
    self.assertIn("server 1", result.lower())
    self.assertIn("5001", result)

  def test_get_version(self):
    result = self.rpc_call(SERVER1, "get_version")
    self.assertIn("3.", result)

  #FILESYSTEM COMMANDS

  def test_make_folder(self):
    foldername = "unit_test_folder"

    # Clean up before test in case it already exists
    if os.path.exists(foldername):
      try:
        os.rmdir(foldername)
```

```
        except:
            pass

    result = self.rpc_call(SERVER1, "make_folder", {"foldername": foldername})
    self.assertTrue("created" in result.lower() or "already exists" in
result.lower())

  def test_search_found(self):
    # Search for a file that should exist in the current folder
    result = self.rpc_call(SERVER1, "search", {"filename": "s.py"})
    self.assertIn("found", result.lower())

  def test_search_not_found(self):
    result = self.rpc_call(SERVER1, "search", {"filename":
"this_file_should_not_exist_123.txt"})
    self.assertIn("not found", result.lower())

  def test_delete_folder(self):
    foldername = "unit_test_delete_folder"

    # Ensure folder exists first
    if not os.path.exists(foldername):
        os.mkdir(foldername)

    result = self.rpc_call(SERVER1, "delete_folder", {"foldername": foldername})
    self.assertIn("deleted", result.lower())

  #Friend/network commands

  def test_online(self):
    result = self.rpc_call(SERVER1, "online", {"port": 5002})
    self.assertIn("online", result.lower())

  def test_list_friends(self):
#Make sure 5002 is known first
    self.rpc_call(SERVER1, "online", {"port": 5002})
    result = self.rpc_call(SERVER1, "list_friends")
    self.assertIn("5002", result)

  def test_heartbeat(self):
    #Make sure 5002 is in friends first
    self.rpc_call(SERVER1, "online", {"port": 5002})
    result = self.rpc_call(SERVER1, "heartbeat")
    self.assertTrue("5002" in result or "pong" in result.lower())

  def test_offline(self):
    # Make sure 5002 is present first
    self.rpc_call(SERVER1, "online", {"port": 5002})
    result = self.rpc_call(SERVER1, "offline", {"port": 5002})
    self.assertIn("offline", result.lower())
```

```
  #ADVANCED COMMANDS

  def test_pass_message(self):
      # Build simple network path: 5001 knows 5002, 5002 knows 5003
      self.rpc_call(SERVER1, "online", {"port": 5002})
      self.rpc_call(SERVER2, "online", {"port": 5003})

      result = self.rpc_call(SERVER1, "pass_message", {"msg": "hello test", "x":
"3"})
      self.assertIn("message received by server 3", result.lower())

  def test_startup(self):
      # This attempts to start server 4
      result = self.rpc_call(SERVER1, "startup", {"x": "4"})
      self.assertIn("started server 4", result.lower())

  def test_shutdown(self):
      #This assumes server 4 has been started first
      self.rpc_call(SERVER1, "startup", {"x": "4"})
      result = self.rpc_call(SERVER1, "shutdown", {"x": "4"})
      self.assertTrue("shutdown signal sent" in result.lower() or "could not contact"
in result.lower())

  def tearDown(self):
      import shutil

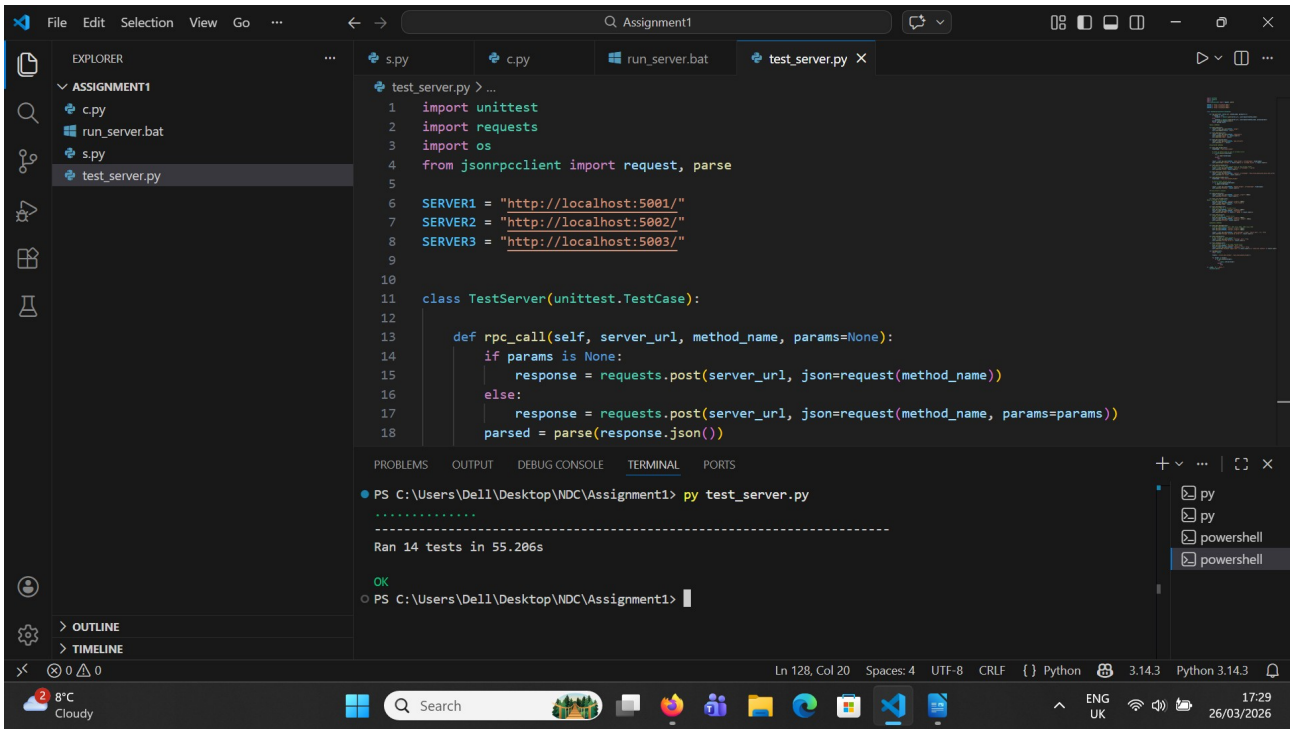
      folders = ["unit_test_folder", "unit_test_delete_folder"]

      for folder in folders:
          if os.path.exists(folder):
              try:
                  shutil.rmtree(folder)
              except:
                  pass

if __name__ == "__main__":
  unittest.main()
```

5. Results of my unit tests :

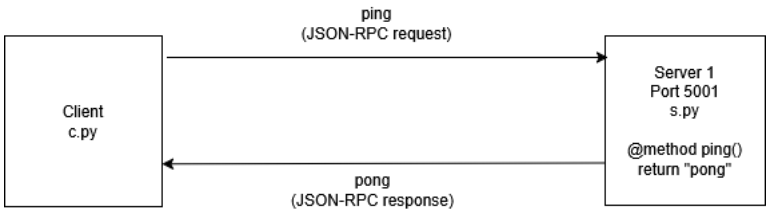
In order to successfully run the file and pass all the unit tests, it's crucial that you first launch server 1, server 2 and server 3 in 3 different terminals, only after that you can write py test_server.py in a new terminal and all unit tests can run properly.



As you can see the result in the terminal, we ran 14 tests which all passed.

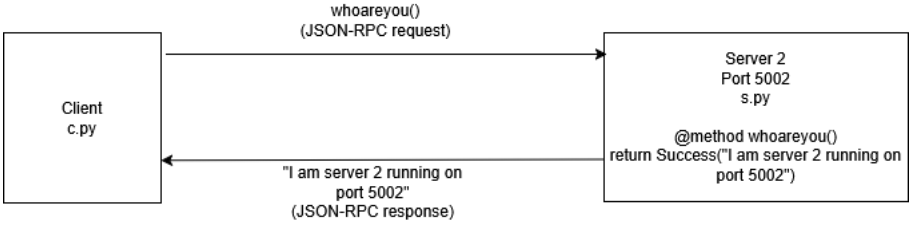
6. All diagrams I've created for the project's functions:

Diagram 1 (ping)



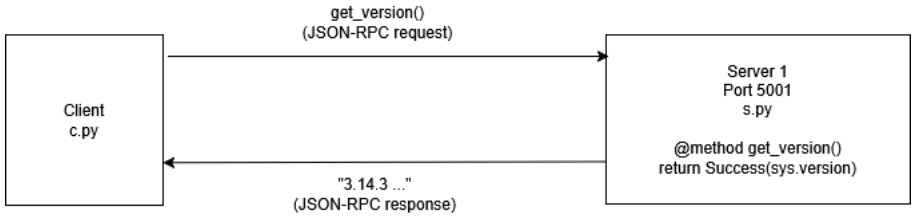
The client sends a ping JSON-RPC request to Server 1 on port 5001, to which server 1 returns a pong JSON-RPC response to the client.

Diagram 2 (whoareyou)



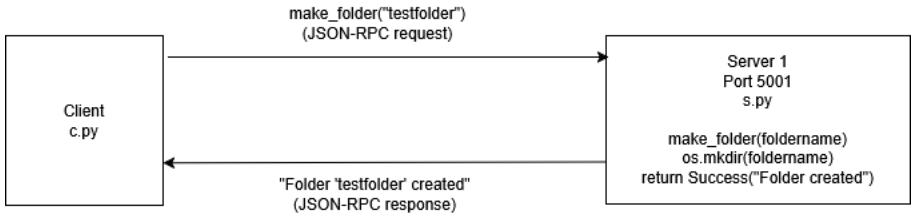
The clients sends a whoareyou() JSON-RPC request to Server 2 on port 5002, to which server 2 returns the server number and port number so the client can identify which server it is communicating to.

Diagram 3 (get_version)



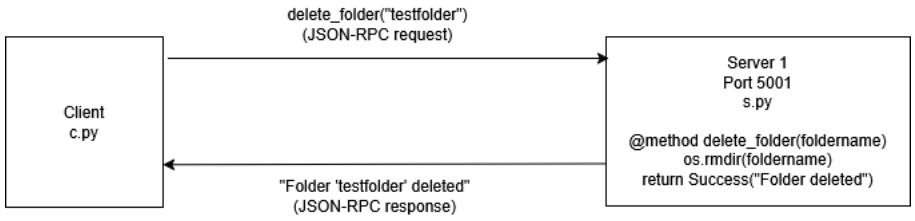
The client sends a get_version() JSON-RPC request to server 1. The server executes the method and returns the python version to the client using sys.version.

Diagram 4 (make_folder)



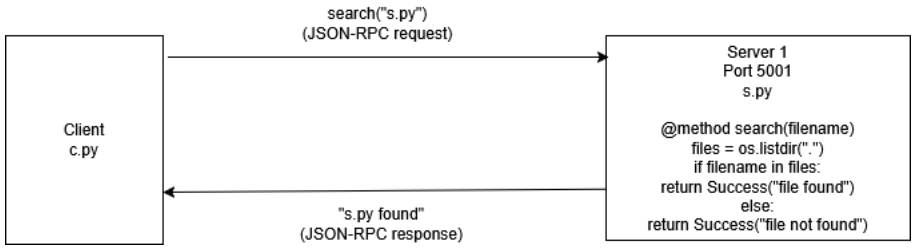
The client sends a JSON-RPC make_folder(“testfolder”) request to server 1. The server executes os.mkdir(foldername) to create the directory and returns a success message to the client.

Diagram 5 (delete_folder)



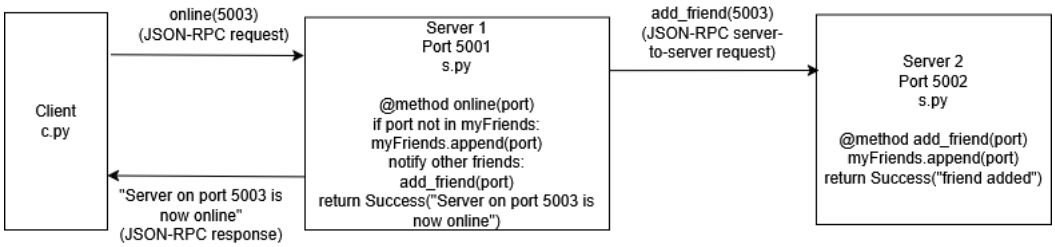
The client sends a JSON-RPC request delete_folder(“testfolder”) to server 1. The server executes os.rmdir(foldername) to delete the directory, and returns a success message to the client.

Diagram 6 (search)



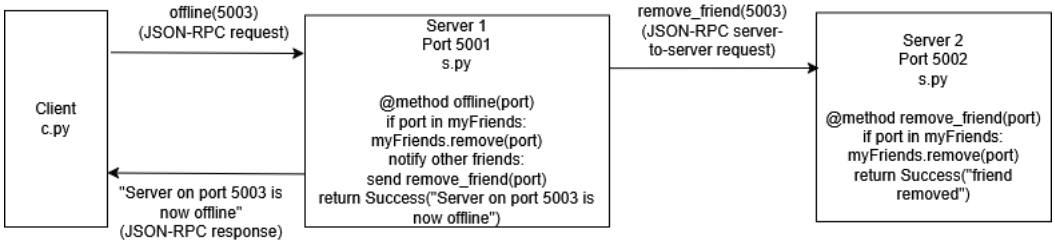
The client sends a JSON-RPC request search(“s.py”) to server 1. The server then reads the list of files using os.listdir() and checks if the file exists. It returns a message if the file was found or if it doesn’t exist.

Diagram 7 (online)



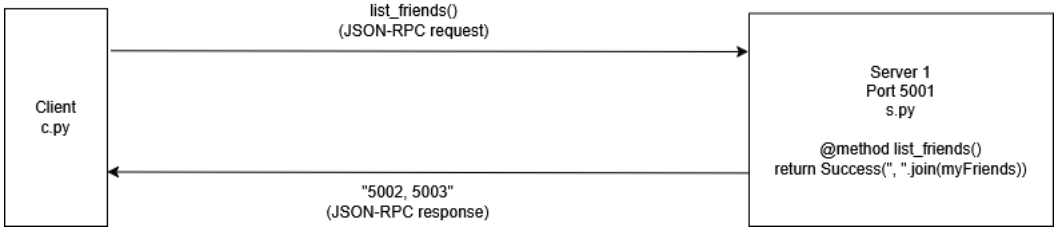
The clients sends online(5003) to server 1 on port 5001. Server 1 adds port 5003 to its friend list and notifies other known servers using the helper command add_friend(5003). It then returns a success message to the client saying that the server on port 5003 is now online.

Diagram 8 (offline)



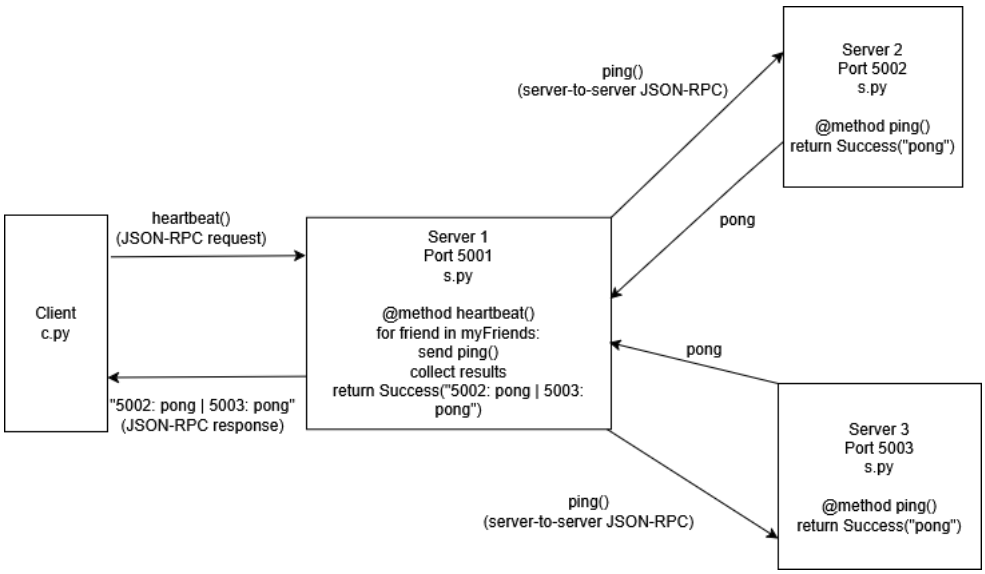
The client sends offline(5003) to server 1 on port 5001. Server 1 removes port 5003 from its friend list and notifies other known servers using remove_friend(5003). It then returns a success message to the client saying that server on port 5003 is now offline.

Diagram 9 (list_friends)



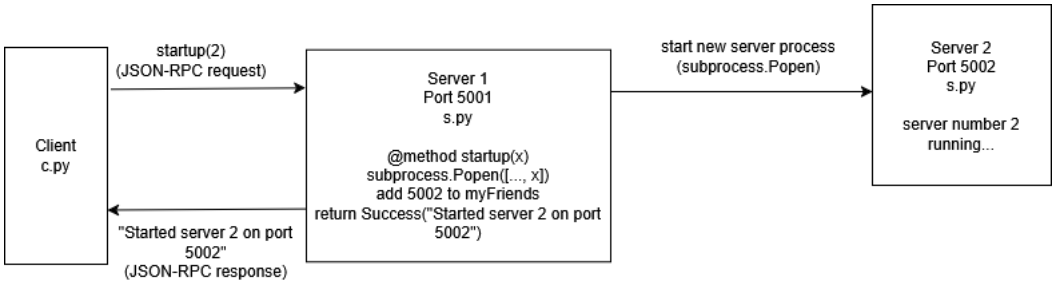
The client sends a list_friends request to the server. The server returns a list of known server ports stored in myFriends.

Diagram 10 (heartbeat)



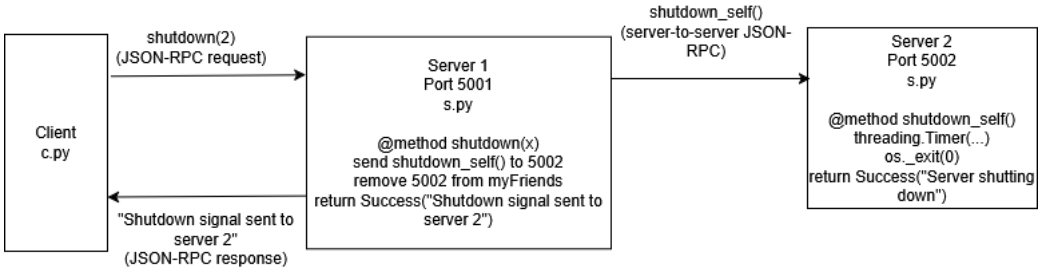
The clients sends heartbeat() to server 1 on port 5001. Server 1 loops through the friends from myFriends list and sends ping() request to the friends. The pong responses are collected and combined into a single status that is then returned to the client.

Diagram 11 (startup)



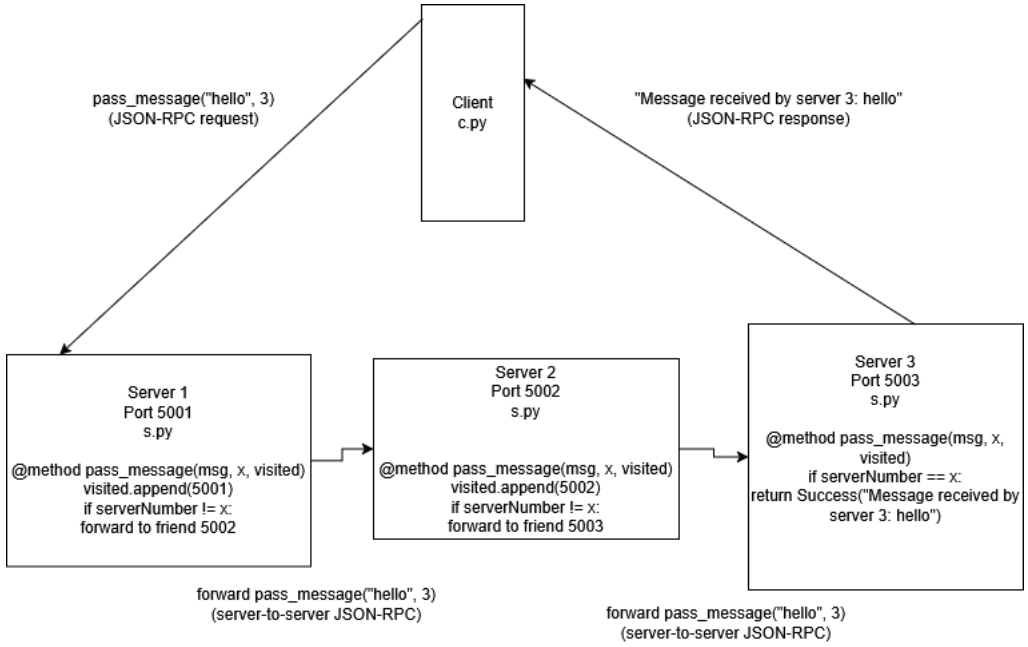
The clients sends startup(2) to server 1 on port 5001. This method tells server 1 to launch server 2 using subprocess.Popen, which starts server 2 on the next available port. Server 1 then updates its friend list and returns a success message to the client saying that server 2 on port 5002 is now live.

Diagram 12 (shutdown)



The client sends shutdown(2) to server 1. Server 1 contacts server 2 on port 5002 using the helper command shutdown_self(). Server 2 then uses os.exit in order to terminate itself, while server 1 updates its friend list and returns a success message to the client.

Diagram 13 (pass_message)



The client sends pass_message(“hello”, 3) to server 1 on port 5001. Server 1 is not the target server, so it appends itself to the visited list and forwards the message to one of its friends. Server 2 repeats the same process and forwards the message to server 3. Since server 3 matches the target server number, it returns a success message to the client, confirming the message received.